

Arquitetura e Desing: Sistema de Gestão de Estoque Farmacêutico

1. Identificação da Equipe

Nome do Integrante	Matrícula
Cristovam Augusto dos Santos Barreiros	2024010392
Diule Monteiro Pereira Junior	2024009300
Eytor Santos Assunção	2024013044
Maria Yasmin Oliveira de Souza	2024019585
Gabriel de Jesus Santiago Cardoso	2025004934

2. Padrão Arquitetural Escolhido: Arquitetura em Camadas(Layered Architecture)

O projeto adota uma Arquitetura em Camadas (Layered Architecture), com forte inspiração nos princípios da Clean Architecture e elementos do padrão Model-View-Controller (MVC) para a API web. Esta escolha é justificada pela necessidade de:

- **Separação de Preocupações (Separation of Concerns):** Cada camada possui uma responsabilidade bem definida, o que facilita a manutenção, o entendimento e a evolução do sistema. Alterações em uma camada (ex: banco de dados) têm impacto mínimo nas outras (ex: lógica de negócio).
- **Testabilidade:** A independência entre as camadas permite testar cada componente isoladamente, garantindo a robustez do código.
- **Flexibilidade e Escalabilidade:** A arquitetura modular permite a substituição de tecnologias em camadas específicas (ex: trocar o banco de dados) sem reescrever todo o sistema. Facilita também a escalabilidade horizontal de componentes.
- **Reusabilidade:** Componentes de camadas mais internas (ex: lógica de negócio) podem ser reutilizados em diferentes interfaces (ex: API, CLI, interface gráfica).

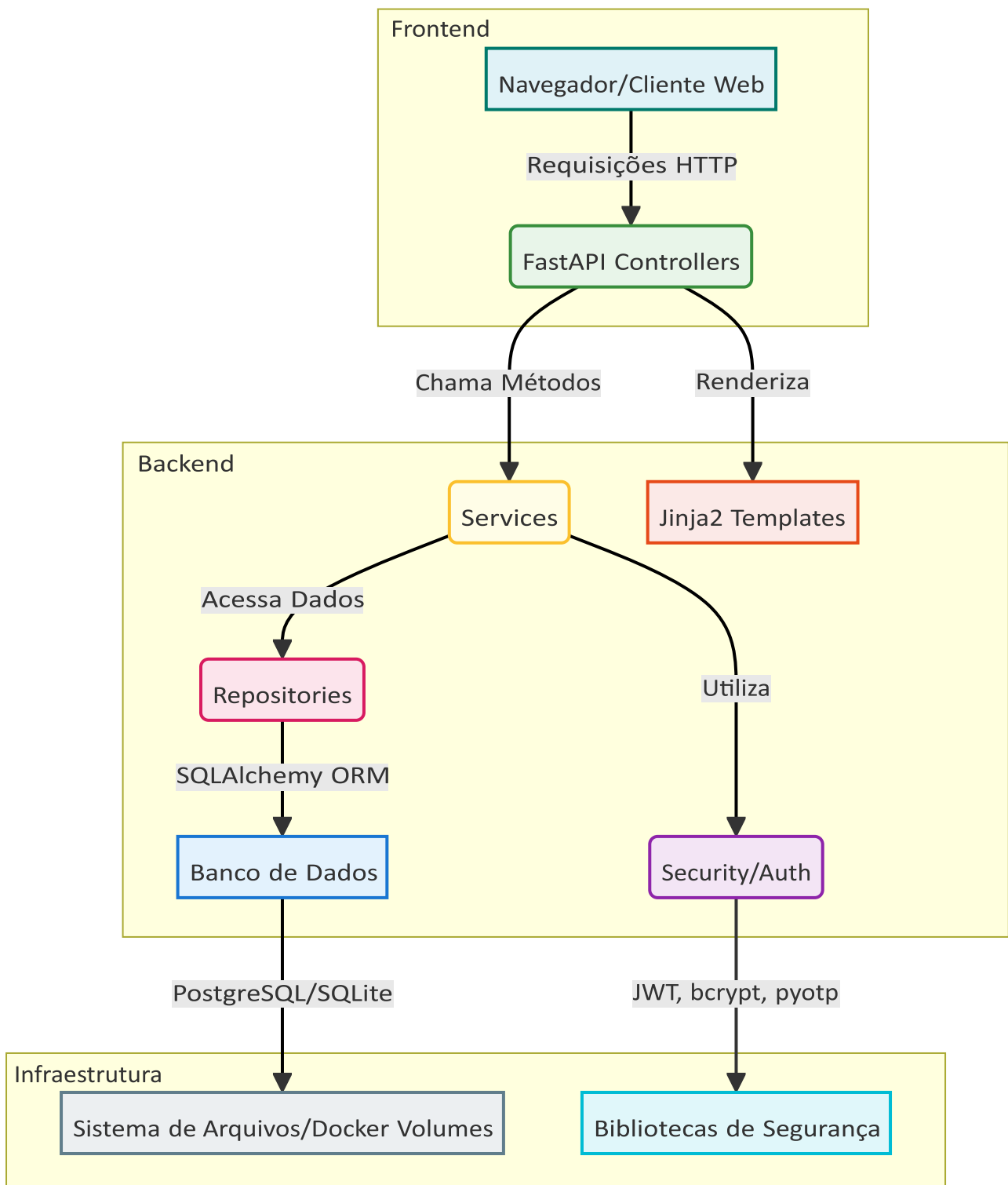
Estrutura das Camadas:

- **Apresentação (Controllers):** Responsável por receber as requisições HTTP, validar os dados de entrada e retornar as respostas. Utiliza o framework FastAPI para definir as rotas e a serialização/desserialização de dados via Pydantic. Também gerencia a renderização de templates HTML (Frontend).
- **Aplicação (Services):** Contém a lógica de negócio principal do sistema. Orquestra as operações, aplica as regras de negócio, coordena as interações entre diferentes entidades e utiliza os repositórios para persistir dados. É a camada que garante a consistência e a integridade dos dados.
- **Domínio (Models):** Define as entidades de negócio (Medicamento, Lote, Usuário, etc.) e suas relações. Representa o “coração” do sistema, sendo independente de qualquer tecnologia de persistência ou interface.

- **Infraestrutura (Repositories, Database, Security):** Lida com detalhes técnicos como persistência de dados (SQLAlchemy ORM), segurança (JWT, 2FA, hashing de senhas) e configurações de ambiente. Os repositórios abstraem a complexidade do banco de dados da camada de serviços.

3. Diagrama de Arquitetura (Componentes)

O diagrama abaixo ilustra a comunicação entre os principais componentes do sistema, refletindo a arquitetura em camadas.



4. Princípios SOLID Aplicados

Os princípios SOLID são fundamentais para a construção de software flexível, manutenível e escalável. Dois exemplos notáveis de sua aplicação no código são:

4.1 Princípio da Responsabilidade Única (SRP – Single Responsibility Principle)

Definição: Uma classe deve ter apenas uma razão para mudar, ou seja, deve ter apenas uma responsabilidade.

Aplicação no Código:

- **app/core/controllers/medication_controller.py** : Esta classe (e outras controllers) tem a única responsabilidade de lidar com as requisições HTTP, rotear para o serviço apropriado e formatar a resposta. Ela não contém lógica de negócio complexa nem acesso direto ao banco de dados.

Exemplo de medication_controller.py

```
@router.post("/", response_model=schemas.MedicationResponse,
status_code=status.HTTP_201_CREATED)
async def create_medication(
    medication: schemas.MedicationCreate,
    medication_service: MedicationService = Depends(get_medication_service),
    current_user: User = Depends(require_stock_manager_or_admin)
):
    """
    Cadastra um novo medicamento no sistema.
    Requer privilégios de Gerente de Estoque ou Administrador.
    """
    db_medication = medication_service.get_medication_by_barcode(medication.barcode)
    if db_medication:
        raise HTTPException(status_code=400, detail="Medicamento com este código de
barras já existe")
    return medication_service.create_medication(medication)
```

- **app/core/services/services.py (ex: MovementService)** : Esta classe é responsável exclusivamente pela lógica de negócio relacionada às movimentações de estoque, incluindo validações complexas e orquestração de outras operações (como a criação de prescrições e atualização de lotes).

Exemplo de MovementService.create_movement em services.py

```
class MovementService(BaseService[MovementRepository]):
    #... (métodos omitidos para brevidade)
    def create_movement(self, movement: MovementCreate, current_user: User) -> Movement:
        #... (validações, lógica de negócio, interação com repositório
        #RF06: Controle de medicamentos controlados (com tarja preta ou vermelha)
        if movement.type == MovementType.EXIT and medication.tarja in
[MedicationTarja.VERMELHA, MedicationTarja.PRETA]:
            #... (lógica de validação de receita e 2FA)
            #... (atualização de quantidade de lote, criação de registro)
        return db_movement
```

4.2 Princípio Aberto/Fechado (OCP – Open/Closed Principle)

Definição: Entidades de software (classes, módulos, funções, etc.) devem ser abertas para extensão, mas fechadas para modificação.

Aplicação no Código:

- **app/core/repositories/base_repository.py** e **app/core/repositories/repositories.py** : A classe `BaseRepository` fornece uma implementação genérica para operações CRUD básicas. Novas entidades (ex: `Fornecedor`) podem ser adicionadas ao sistema criando-se um novo modelo e um novo repositório que herda de `BaseRepository`, sem a necessidade de modificar o código da `BaseRepository`.

Exemplo de base_repository.py

```
class BaseRepository(Generic[ModelType]):
    def __init__(self, model: Type[ModelType], db: Session):
        self.model = model
        self.db = db
# ... (métodos genéricos como get, get_multi, create, update, delete)
```

Exemplo de repositories.py

```
class MedicationRepository(BaseRepository[Medication]):
    def __init__(self, db: Session):
        super().__init__(Medication, db)
```

Isso significa que o comportamento de acesso a dados pode ser estendido para novas entidades sem alterar o código existente da `BaseRepository`.

5. Design Pattern Utilizado: Repository Pattern

Padrão: Repository Pattern

Justificativa: O Repository Pattern é utilizado para abstrair a camada de persistência de dados da lógica de negócio. Ele fornece uma interface limpa para a camada de serviços interagir com o banco de dados, sem se preocupar com os detalhes de implementação (SQLAlchemy, SQL puro, etc.).

Aplicação no Código:

- **app/core/repositories/base_repository.py** : Define a interface genérica para as operações de repositório (CRUD).
- **app/core/repositories/repositories.py** : Contém as implementações concretas dos repositórios para cada entidade (ex: `MedicationRepository`, `UserRepository`).
- **app/core/services/services.py** : As classes de serviço recebem instâncias de repositórios via injeção de dependência, utilizando-os para realizar operações de persistência.

Benefícios:

- **Desacoplamento:** A lógica de negócio nos serviços não depende diretamente do ORM ou do banco de dados, facilitando a troca da tecnologia de persistência.
- **Testabilidade:** Permite a criação de mocks ou fakes dos repositórios em testes unitários dos serviços, sem a necessidade de um banco de dados real.

- **Centralização:** A lógica de acesso a dados é centralizada nos repositórios, evitando duplicação de código e garantindo consistência.

6. Tecnologias Utilizadas

A escolha das tecnologias foi baseada na performance, produtividade do desenvolvedor, robustez e capacidade de atender aos requisitos do MVP.

Categoria	Tecnologia	Justificativa
Backend Framework	FastAPI	Framework web moderno e de alta performance para construção de APIs assíncronas em Python. Oferece validação de dados automática com Pydantic, documentação interativa (Swagger UI/ReDoc) e tipagem forte, acelerando o desenvolvimento e reduzindo erros.
ORM (Object-Relational Mapper)	SQLAlchemy 2.0	Biblioteca robusta e flexível para interação com bancos de dados relacionais em Python. Permite trabalhar com objetos Python em vez de SQL puro, facilitando a manipulação de dados e o gerenciamento de transações.
Validação de Dados	Pydantic V2	Utilizado para definir esquemas de dados (modelos) e realizar validação automática de entrada e saída de dados. Integrado nativamente ao FastAPI, garante a consistência dos dados que trafegam pela API.
Autenticação/Autorização	JWT (JSON Web Tokens)	Padrão seguro para troca de informações entre partes como um token. Permite autenticação stateless, ideal para APIs RESTful.

Categoria	Tecnologia	Justificativa
Hashing de Senhas	bcrypt	Algoritmo de hashing de senhas seguro e resistente a ataques de força bruta, garantindo a proteção das credenciais dos usuários.
Autenticação de Dois Fatores (2FA)	pyotp	Biblioteca Python para geração e verificação de códigos TOTP (Timebased One-Time Password), implementando o 2FA para maior segurança.
Banco de Dados	PostgreSQL / SQLite	PostgreSQL é um banco de dados relacional open-source avançado, robusto e escalável, ideal para ambientes de produção. SQLite é usado para desenvolvimento local e testes devido à sua simplicidade e portabilidade (banco de dados em arquivo).
Migrações de Banco de Dados	Alembic	Ferramenta para gerenciar migrações de esquema de banco de dados com SQLAlchemy, permitindo evoluir o esquema de forma controlada e reversível.
Servidor ASGI	Uvicorn	Servidor ASGI de alta performance para aplicações Python, otimizado para FastAPI, garantindo respostas rápidas às requisições.

Categoria	Tecnologia	Justificativa
Frontend (Templates)	Jinja2	Motor de templates rápido e expressivo para Python, utilizado para renderizar as páginas HTML da interface do usuário.
Frontend (Linguagens)	HTML5, CSS3, JavaScript Vanilla (ES6+)	Tecnologias padrão da web para construção de interfaces de usuário. A escolha de JavaScript Vanilla minimiza dependências e mantém o projeto leve e performático.
Orquestração de Contêineres	Docker / Docker Compose	Ferramentas para containerização e orquestração de aplicações, garantindo ambientes de desenvolvimento e produção consistentes e facilitando o deploy.